

1. Deep research para defender TFG_CYBER_AI

1.1. Qué he decidido investigar

He dejado fuera, como pediste, la investigación interna de `sb3_contrib` y de QR-DQN. Tras inspeccionar el repositorio en su estado actual sobre `main`, lo más importante para tu defensa no es volver a explicar el algoritmo, sino dominar el **contrato de datos** del proyecto, el **adapter de CICIDS2017**, la **formalización del problema como entorno RL**, el **pipeline de entrenamiento y reproducibilidad**, la **validación** y la **Fase 2 de inferencia sobre tráfico real**. Esos son, de hecho, los entry points y los invariantes que el propio repo declara como “source of truth”: `src/canonical_schema.py`, `src/load_cicids2017.py`, `src/rl_defender_env.py`, `src/train_rl_defender.py`, `src/validate_ch`, `src/validate_leave_one_csv_out.py` y `scripts/predict_real_traffic_v2.py`.

Mi lectura está orientada a que puedas responder no solo *qué has usado*, sino **por qué existe cada pieza, qué hipótesis codifica, qué garantiza, qué no garantiza y qué limitaciones honestas tiene hoy el sistema**. El foco correcto, por tanto, no es “tengo un modelo que clasifica bien”, sino “tengo un pipeline reproducible y defendible desde el dataset hasta el artefacto de Fase 2”. Eso también coincide con las notas de defensa ya presentes en el repo.

1.2. Esquema canónico y contrato de datos

La decisión arquitectónica más importante del proyecto no es el algoritmo, sino el **espacio de observación estable**. El dataset oficial CICIDS2017, publicado por el Canadian Institute for Cybersecurity, ofrece PCAPs y CSVs con **más de 80 features de flujo** extraídas con CICFlowMeter, a lo largo de cinco días de tráfico normal y malicioso. Tu repo no usa “todo lo que haya”, sino que selecciona **76 features canónicas flow-based**, deliberadamente numéricas, estables y extraíbles también desde tráfico real; además excluye IPs, timestamps, Flow IDs y puertos específicos por riesgo de leakage o de representar atajos espurios hacia la etiqueta.

Ese diseño produce un vector final de **152 dimensiones**: las 76 features canónicas y otras 76 posiciones de máscara de missingness. La idea de añadir indicadores binarios de ausencia es sólida desde el punto de vista de ingeniería de datos: en aprendizaje automático, los **missing indicators** se usan precisamente para no confundir “valor imputado” con “valor observado”, de modo que el modelo pueda explotar el patrón de ausencia como señal adicional. En tu código, la semántica declarada es `1 = present/valid` y `0 = missing/imputed`.

Aquí hay una sutileza muy importante para tu defensa: **mi lectura del código sugiere que, en CICIDS2017, la máscara no conserva todos los NaN originales fila a fila**. El motivo es que `load_cicids2017.py` reemplaza `inf` por `NaN` y luego rellena las features con 0 antes de llamar a `map_to_canonical()`. Después, `map_to_canonical()` marca la máscara como 0 solo si todavía ve valores no finitos. Eso significa que, en la práctica del pipeline actual, la máscara parece servir sobre todo para **ausencias estructurales** entre datasets o extractores, no para preservar toda la missingness original de cada celda de CICIDS2017. Es una observación muy defendible si te preguntan “¿la máscara codifica missingness real del dataset o compatibilidad de esquema?”; la respuesta correcta hoy es: **principalmente compatibilidad de esquema y extractor, según el flujo actual de preprocesado**.

Otra sutileza todavía más fina: el **StandardScaler** se ajusta sobre el vector completo de 152 dimensiones, no solo sobre las 76 features reales. Eso implica que la máscara entra al modelo **escalada**, no como bits crudos 0/1. Conceptualmente, esto transforma la señal “presente/ausente” en “presente/ausente respecto al patrón estadístico de train”. Si en entrenamiento una mask feature es casi siempre 1, tras escalar tenderá a 0; si en inferencia aparece un 0 donde antes casi nunca lo había, entrará como una desviación clara. En otras palabras: aunque sobre el papel la máscara sea binaria, **a la red le llega como feature estandarizada**, y eso refuerza su papel como detector de mismatch estructural.

La defensa verbal más fuerte aquí es esta: **no has diseñado un modelo acoplado a un CSV concreto**, sino un contrato de observación estable entre datasets históricos y tráfico real. Esa es, probablemente, la decisión más “de ingeniería de sistema” de todo el TFG.

1.3. Carga de CICIDS2017 y preprocesado

El adapter de CICIDS2017 (`src/load_cicids2017.py`) está más trabajado de lo que parece a simple vista. Reconoce explícitamente los **8 CSVs oficiales** del dataset, lee por *chunks* para no romper RAM, localiza la columna de etiqueta de forma robusta, binariza **BENIGN** frente a **ATTACK**, y elimina columnas tipo identificador como **Flow ID**, **Timestamp**, **IPs** y **puertos**. Esta eliminación no es cosmética: scikit-learn recuerda que hay leakage cuando el modelo aprende con información que no estará disponible —o no debería influir causalmente— en el momento real de predicción; los identificadores y proxies temporales son candidatos clásicos a ese problema.

Tienes tres formas de particionar CICIDS2017, y deberías saber defender las tres. La primera es el **random stratified split**, útil para iteración rápida y para cuantificar rendimiento offline bajo distribución similar. La segunda es el **day/CSV split**, donde se entrena en unos días/CSV y se evalúa en otros; esto es más realista porque el dataset oficial está organizado por días con condiciones y ataques distintos. La tercera es el **leave-one-exact-CSV-out**, que deja fuera exactamente uno de los CSV reales y entrena sobre los otros siete. Esa tercera vía es metodológicamente la más fina porque reduce el riesgo de que el modelo “vea” patrones demasiado parecidos entre train y test.

La lógica del day split se entiende mejor si recuerdas cómo está construido CICIDS2017: la página oficial describe cinco días de captura, con **lunes de actividad normal** y **martes a viernes con tráfico benigno y ataques**. Por eso tiene sentido que el repo use, por defecto, lunes-martes-miércoles para train y jueves-viernes para test en el modo “day”: no es un capricho, sino un intento de medir generalización a patrones temporales distintos. Si te preguntan “¿por qué no basta con un split aleatorio?”, la respuesta fuerte es: **porque un split aleatorio puede sobreestimar rendimiento cuando train y test comparten distribuciones demasiado parecidas; el split por día/CSV es más exigente y más cercano al escenario de despliegue**.

También es correcta la forma en que el repo escala los datos: el **StandardScaler** se ajusta sobre train y después se aplica a test, justo como recomiendan las guías de buenas prácticas de scikit-learn para evitar contaminación entre subconjuntos. Además, la documentación oficial de **StandardScaler** advierte que este transformador es **sensible a outliers**, un punto crucial para entender por qué en Fase 2 aparecen percentiles y clipping en z-score.

1.4. Entorno RL y qué problema está resolviendo realmente

Tu entorno `RLDatasetDefenderEnv` es conceptualmente simple y, precisamente por eso, hay que saber explicarlo muy bien. La observación es la muestra actual $X[i]$. La acción es binaria: 0 = PERMIT, 1 = BLOCK. La recompensa se calcula **solo** a partir de la etiqueta real de esa muestra y de la acción elegida; después el entorno avanza a la siguiente muestra. No hay estado latente de red, no hay evolución del atacante en respuesta a la acción del agente, no hay dinámica compleja de sistema. Lo que hay es una secuencia de decisiones por flujo con coste asimétrico.

Eso te lleva a una respuesta muy importante si el tribunal te pregunta “¿esto es de verdad RL o es clasificación disfrazada?”. La respuesta honesta es: **en la implementación actual, el problema se parece mucho a un contextual bandit o a una clasificación secuencial con coste**, porque en cada paso el agente recibe contexto (el flujo), elige acción y recibe recompensa basada en contexto+acción. Esa es, de hecho, una definición estándar de contextual bandit en la literatura. La parte defendible a favor de RL no está en decir que hoy ya modelas una red interactiva completa, sino en decir que **la formulación por recompensa te permite incorporar directamente la asimetría de costes de ciberseguridad y deja abierta la evolución futura hacia bloqueo activo y decisiones realmente secuenciales**.

La recompensa actual del repo es `tp=1.5`, `fp=-1.5`, `fn=-5.0`, `omission=0.0`, y esto está alineado en entrenamiento, validación y entorno. Pero aquí hay otra sutileza de código que conviene dominar: aunque el docstring del entorno todavía habla de un posible `tn`, **la implementación ejecutada no usa una recompensa separada para true negative**. Cuando el tráfico es benigno y la acción es PERMIT, el retorno es `omission`, que por defecto vale 0.0. Es decir, el agente recibe una fuerte penalización por dejar pasar ataques, otra penalización por bloquear benignos, una recompensa moderada por bloquear ataques, y neutralidad por permitir benignos. En términos de diseño, eso prioriza no cometer falsos negativos y no convierte el PERMIT correcto en una “recompensa positiva”, sino en ausencia de castigo. Debes poder explicarlo así, porque es exactamente la lógica codificada.

También debes conocer la mecánica de episodios. En entrenamiento, `train_rl_defender.py` fija `max_steps_ep = min(10_000, len(X_train))`, y cada `reset()` baraja los índices. Eso significa que un episodio no equivale necesariamente a “recorrer entero el dataset”; más bien, el entrenamiento consume **segmentos aleatorizados** del conjunto de train. En evaluación, en cambio, `shuffle=False` y el test se recorre en orden determinista. Y Check A, además, elimina por completo la dependencia del entorno porque compara `model.predict(X_test[i])` directamente contra `y_test[i]`. Esto es importante para defender que las métricas no dependen de una magia interna del `env.step()`.

Finalmente, hay dos piezas de *framework plumbing* que debes saber justificar. `DummyVecEnv` existe porque Stable-Baselines3 trabaja con entornos vectorizados y, según su propia documentación, puede usarse incluso cuando solo quieres entrenar con un único entorno. `Monitor`, por su parte, envuelve el entorno para registrar recompensas, duración y tiempo de episodio. Eso no es “decoración”: es parte de la trazabilidad experimental.

1.5. Entrenamiento, artefactos y reproducibilidad

El script de entrenamiento actual hace varias cosas bien desde el punto de vista de reproducibilidad. Primero, crea un `RUN_ID` con timestamp y lo usa para organizar salidas. Segundo, carga CICIDS2017 **sin escalar** al principio para poder calcular percentiles sobre train y persistir el `scaler` correctamente. Tercero, guarda no solo el `.zip` del modelo, sino

también `config.json`, `metrics.json`, `scaler.joblib` y `train_percentiles.npz`. Eso convierte cada experimento en un artefacto autocontenido y reutilizable en Fase 2.

Hay además dos detalles finos que conviene saber. El primero es que el script calcula percentiles $p_{0.5}$ y $p_{99.5}$ **solo sobre las primeras 76 dimensiones canónicas**, no sobre las 152. Esto es deliberado: el clipping robusto se reserva para las features continuas, no para la máscara. El segundo es que el *help text* del parser está algo desalineado con el código ejecutado: el comentario de ayuda habla de defaults antiguos, pero `main()` resuelve hoy 25_000 timesteps en modo fast y 100_000 en modo full cuando no pasas `-timesteps`. Si te preguntan por defaults de entrenamiento, apóyate siempre en **lo que hace el código**, no en el comentario heredado.

El mejor artefacto comprometido del repo es `C03_qrdqn_cicids2017_canonical_full_random_202`. Su `config.json` indica 500 000 filas máximas cargadas, 100 000 timesteps, 152 features, arquitectura `[512, 256]`, `batch_size = 2048`, `gradient_steps = 20`, dispositivo cuda y una `reward_config` histórica de `tp = 1.5`, `fp = -2.0`, `fn = -5.0`, `omission = 0.0`. Su `metrics.json` recoge `accuracy = 0.99859`, `recall_attack = 0.99945` y `f1_attack = 0.99876`.

Ese punto enlaza con otra respuesta de defensa imprescindible: **los defaults actuales del código no son idénticos a la configuración del mejor run histórico**. El proyecto documenta hoy `fp = -1.5` como normalización vigente, pero el mejor artefacto histórico usó `fp = -2.0`. Si el tribunal te pregunta sobre los rewards que usaba tu mejor modelo, no contestes con los defaults actuales; contesta con los del artefacto concreto. Y si te preguntan sobre lo que usa hoy el código, entonces sí: `tp=1.5`, `fp=-1.5`, `fn=-5.0`, `omission=0.0`. Esa distinción entre **estado actual del código** y **estado histórico del run** es uno de los mensajes más maduros metodológicamente en tu repo.

1.6. Validación y lectura correcta de métricas

Tu suite de validación no es un adorno; es una de las mejores partes defendibles del proyecto. `validate_checks.py` implementa tres chequeos con propósitos muy distintos. **Check A** evalúa el modelo de forma directa, sin depender del `info["true_label"]` del entorno. **Check B** reentrena con etiquetas barajadas para detectar leakage. **Check C** entrena y evalúa sobre CSVs/días distintos para medir generalización dura. Además, aparte, tienes `validate_leave_one_csv_out.py`, que implementa la validación leave-one-exact-CSV-out por CSV oficial.

Check A sirve para responder a la objeción “a lo mejor tu accuracy depende de cómo el entorno informa la etiqueta”. El artefacto versionado `VAL_checks_A_20260212_235443` muestra `accuracy = 0.9939`, `TP = 4772`, `FP = 60`, `FN = 1` y `TN = 5167`, usando exclusivamente predicción directa sobre `X_test`.

Check B es metodológicamente todavía más fuerte. El artefacto `VAL_checks_B_20260212_235736` reporta `shuffled_accuracy = 0.4773`, por debajo del baseline de clase mayoritaria 0.5227, y deja `leakage_detected = false`. La lectura correcta es que **cuando destruyes la relación entre features y etiquetas, el rendimiento no se mantiene artificialmente alto**. Eso es justo lo que quieres ver si no hay leakage.

Check C es donde el proyecto se vuelve realmente honesto. El artefacto `VAL_checks_C_20260213_004` baja a `accuracy = 0.84135`, `precision_attack = 0.76479`, `recall_attack = 0.52954` y `f1_attack = 0.62578`, con train en Monday/Tuesday/Wednesday y test en Thursday/Friday. Ese salto frente al mejor resultado aleatorio (0.99859) es exactamente el tipo de evidencia que demuestra que no te has quedado en un benchmark cómodo: el pipeline funciona muy bien *in-distribution*, pero generalizar a días/CSV distintos es bastante más duro.

La validación leave-one-exact-CSV-out es, conceptualmente, aún mejor para discutir robustez porque fuerza al modelo a no ver nunca un CSV exacto durante entrenamiento. Pero aquí la defensa honesta importa más que el entusiasmo: el repo declara que el flujo está implementado, aunque **todavía no hay un artefacto completo comprometido** bajo `runs/validation/` para reportar métricas consolidadas. Debes presentarlo como una validación ya soportada por el código, no como un resultado ya cerrado.

1.7. Fase 2, inferencia robusta y domain shift

La Fase 2 del proyecto no es “despliegue en producción”, sino **inferencia offline robusta sobre tráfico capturado en un laboratorio privado**. El propio repo lo deja claro: el alcance actual es capturar tráfico, extraer flujos, mapearlos al esquema canónico, ejecutar inferencia y guardar artefactos reproducibles; quedan fuera el bloqueo activo en línea, la exposición pública del laboratorio y el despliegue operacional a gran escala.

La pieza central aquí es `predict_real_traffic_v2.py`, y debes saber describirla paso a paso. El script carga un `flows.csv`, separa metadata opcional (`src_ip`, `dst_ip`, puertos, protocolo, timestamp y posibles columnas de verdad terreno), intenta armonizar unidades temporales convirtiendo segundos a microsegundos cuando detecta medianas demasiado pequeñas, mapea las columnas del extractor real a los 76 nombres canónicos, recompone el vector de 152 dimensiones con máscara, aplica **percentile clipping** opcional sobre las primeras 76 dimensiones, luego el `StandardScaler` persistido, luego **z-score clipping** opcional, calcula diagnósticos de distribución, carga el modelo y predice por lotes para evitar problemas de memoria. Finalmente guarda `predictions.csv`, `metrics.json` y `config.json`, y opcionalmente `diagnostics.json`.

La lógica detrás de ese endurecimiento es muy defendible. `StandardScaler` es sensible a outliers, algo que la documentación oficial de scikit-learn dice explícitamente, y el propio campo de *dataset shift* estudia precisamente qué pasa cuando la distribución de entrada en test o despliegue difiere de la de entrenamiento. El repo introduce dos mitigaciones concretas: limitar las features brutas a los percentiles entrenados ($p_{0.5}$, $p_{99.5}$) y truncar después las features escaladas a un máximo $|z|$. En términos intuitivos: primero evitas que un flujo real absurdamente extremo rompa la escala; después evitas que una z-score extrema empuje la red a un régimen muy fuera de distribución.

Los artefactos comprometidos de Phase 2 muestran por qué esta parte todavía es científicamente “abierta”. En `P2v2_pred_20260224_004121`, sobre `pcaps/flows_benign.csv`, el sistema registró `block_rate = 1.0` y el `config.json` indica que **no** se usaron percentiles (`"percentiles": null`). En el artefacto posterior `P2v2_pred_20260408_230318`, con el mismo modelo y el mismo `scaler`, el `block_rate` pasó a `0.0`, y esta vez sí consta el uso de `train_percentiles.npz`. No debes sobrerreaccionar diciendo “los percentiles arreglaron por sí solos la Fase 2”, porque el repo no demuestra causalidad estricta entre ambos hechos; pero sí puedes afirmar con rigor que la conducta de inferencia sobre tráfico benigno real **ha cambiado sensiblemente entre artefactos**, y que una diferencia visible entre ambos runs es precisamente el uso de clipping por percentiles.

Esa es, de hecho, la lectura metodológica correcta de la Fase 2: **el pipeline técnico está ya bien montado; lo que sigue abierto es la robustez frente a domain shift**.

1.8. Preguntas probables del tribunal

¿Por qué 76 features y no todas las del CSV original? Porque el repo prioriza un contrato de observación que sea *flow-based*, numérico, estable y reutilizable en tráfico real, y por eso elimina identificadores y proxies de leakage como IPs, timestamps, Flow IDs y puertos específicos.

¿Para qué sirve exactamente la máscara de missingness? Sirve para distinguir “he imputado/esta feature no estaba disponible” de “he observado un valor real”. En la implementación actual, su utilidad principal está en **mismatch estructural entre datasets o extractores**, especialmente en Fase 2.

¿Por qué RL y no clasificación supervisada pura? La respuesta honesta es que la implementación actual se parece bastante a un **contextual bandit** o a clasificación secuencial con costes. RL sigue siendo defendible porque permite optimizar directamente una función de utilidad asimétrica de ciberseguridad y deja el marco preparado para una evolución futura hacia decisiones realmente secuenciales y bloqueo activo.

¿Qué optimiza de verdad el sistema de recompensas? Optimiza una asimetría de seguridad: castiga mucho más dejar pasar ataques ($fn = -5.0$) que bloquear benignos ($fp = -1.5$), recompensa bloquear ataques ($tp = 1.5$) y deja neutro permitir benignos ($omission = 0.0$).

¿Por qué no basta con la accuracy del mejor modelo? Porque el mejor resultado comprometido (0.99859) viene de un split aleatorio sobre la misma distribución; cuando pasas a una validación por días/CSV distintos, la accuracy baja a 0.84135 y el recall de ataque a 0.5295.

¿Cómo has intentado evitar leakage? De tres formas: quitando features potencialmente espurias; ajustando el `scaler` solo sobre train; y ejecutando un shuffled-label test donde el rendimiento no se mantiene artificialmente alto.

¿Qué parte está realmente resuelta y cuál no? Está resuelta la arquitectura experimental: contrato de datos, adapter, entorno, entrenamiento reproducible, validación A/B/C y pipeline robusto de Fase 2. No está resuelto todavía el **bloqueo activo en tiempo real** ni la **robustez cerrada frente a tráfico real benigno y mixto**.

¿Cuál es la formulación más sólida para cerrar la defensa? Que tu TFG no aporta solo “un modelo que clasifica”, sino un **pipeline de defensa RL reproducible y trazable**, con un contrato de observación coherente, control explícito del leakage, validación dura y una transición ya operativa hacia tráfico real offline.